TXT Files in Python

- **Text Files Overview**:
  - Text files contain only plain text and are often used to store data.
  - Common file extension: .txt
  - Programs expect text files to follow a specific format.

- **Python** open() **Function**:
  - Used to work with text files in Python by returning a file object.
  - Syntax: f = open(filename, mode)
  - Common modes:
    - 'r': Read mode (default).
    - 'w': Write mode (creates a new file if not existing).
    - 'a': Append mode.
    - 'r+': Read and write mode.
    - 'w+': Write and read, discarding previous content.
    - 'a+': Append and read mode.

- **Best Practice**: Always close files after use with f.close() to avoid data loss.

- **Writing to a File**:
  - Example:

```python
f = open('test.txt', 'w')
for i in range(5):
    f.write(f"This is line {i}\n")
f.close()
```

- Writes lines to test.txt and saves them.

- **Appending to a File**:
  - Example:

```python
f = open('test.txt', 'a')
f.write("This is another line\n")
f.close()
```

- **Reading from a File**:
  - Reading entire content:

```python
f = open('test.txt', 'r')
content = f.read()
f.close()
```

- Reading line by line:

```python
f = open('test.txt', 'r')
contents = f.readlines()
f.close()
```

Working with Numbers and Arrays in TXT Files

- **Saving Numerical Data**:
  - Using numpy to handle arrays:

```python
import numpy as np
arr = np.array([[1.20, 2.20, 3.00], [4.14, 5.65, 6.42]])
np.savetxt('my_arr.txt', arr, fmt='%.2f', header='Col1 Col2 Col3')
```

  - Saves a 2D array to my_arr.txt with a custom header.

- **Loading Numerical Data**:
  - Reading arrays back into Python:

```python
my_arr = np.loadtxt('my_arr.txt')
```

- Automatically skips the header and loads the array back into memory.

CSV Files in Python

- **What are CSV Files?**
  - CSV stands for *Comma-Separated Values*, a simple and widely used file format for storing tabular data.
  - Each line in a CSV file represents a data record, and each record consists of one or more fields separated by commas.
  - CSV files are highly compatible with applications like *Microsoft Excel*, making them easy to visualize.

- **Python's CSV Handling:**
  - Python provides several tools to handle CSV files, including the csv module and third-party libraries like numpy and pandas.
  - In scientific applications, CSV files often store large tables of numerical data, which can be efficiently managed using Python.

Writing and Reading CSV Files Using Numpy

- **Writing a CSV File:**
  - Python's numpy package can be used to write CSV files efficiently.
  - Example code to generate and save random data to a CSV file:

```python
import numpy as np
data = np.random.random((100, 5))
np.savetxt('test.csv', data, fmt='%.2f', delimiter=',', header='c1, c2, c3, c4, c5')
```

- The code creates a file with 100 rows and 5 columns of random numbers, saved in CSV format.

- **Reading a CSV File:**
  - The same numpy package allows reading CSV files into arrays for easy data manipulation:

```python
my_csv = np.loadtxt('test.csv', delimiter=',')
print(my_csv[:5, :])
```

  - This reads the CSV back into a numpy array and prints the first five rows.

Beyond Numpy for CSV Handling

- **Advanced CSV Handling with Pandas:**
  - While numpy works well for numerical data, pandas provides more advanced capabilities, especially for working with labeled data and DataFrames.
  - pandas supports easier manipulation of CSV data, including data cleaning, filtering, and summarization.

  - Example code to use pandas for CSV operations:

```python
import pandas as pd
df = pd.read_csv('test.csv')
print(df.head())
```

  - This provides a richer interface for working with CSV files, especially for tabular data analysis.

Pickle Files

Overview:

- **Pickle** is a Python module used for **serializing** and **de-serializing** Python object structures.
- Serialization involves converting objects in memory (e.g., dictionaries, lists) into a **byte stream** for storage on disk as a **binary file**.
- De-serialization is the reverse process, where the byte stream is converted back into its original object form.

When to Use:

- Useful for saving Python-specific data types, like dictionaries, lists, or tuples, for later use.
- Ideal for sharing data with other Python programs but has limitations when sharing across different programming languages.

Key Functions:

- pickle.dump(): Serializes an object and writes it to a file.
- pickle.load(): Reads a pickle file and de-serializes it back into the original Python object.

Pickling and Unpickling Process

How to Write a Pickle File:

- First, import the pickle module: import pickle.
- Example of serializing a dictionary:

```python
dict_a = {'A':0, 'B':1, 'C':2}
pickle.dump(dict_a, open('test.pkl', 'wb'))
```

- The file is saved in **binary** format ('wb' stands for write-binary mode).

How to Read a Pickle File:

- De-serialize the binary file back to a Python object:

```python
my_dict = pickle.load(open('./test.pkl', 'rb'))
```

- The mode 'rb' signifies read-binary mode.

Limitations and Compatibility

Compatibility Issues:

- **Not a universal format**: While TXT and CSV files can be used across languages (e.g., R, MATLAB, Java), Pickle is specific to Python.
- **Python Version Differences**: Pickle files created in Python 2 may not be compatible with Python 3. Use encoding='latin1' in the pickle.load() function for compatibility with Python 2 files.

Example:

- Reading a Python 2 Pickle file in Python 3:

```python
new_dict = pickle.load(open(filename, 'rb'), encoding='latin1')
```

Key Warning:

- Avoid using Pickle files if you need cross-language compatibility or long-term data preservation, as formats may change across Python versions.

JSON Files in Python

- **What is JSON?**
  - JSON stands for JavaScript Object Notation.
  - A lightweight, language-independent data format often used for data exchange.
  - JSON files typically end with the extension .json.

- **Key Characteristics**
  - Stores data in key-value pairs, similar to Python dictionaries.
  - Supports various data types: strings, numbers, lists, and nested objects.
  - Compact and faster than Pickle for serialization (Python object to byte stream).

- **Why Use JSON?**
  - Language agnostic: Can be read by many programming languages.
  - More efficient disk usage compared to Pickle.
  - JSON is ideal for cross-platform data exchange, making it widely adopted in APIs.

Working with JSON Files in Python

- **Writing to a JSON File**
  - Use Python's json library to easily handle JSON.
  - Example:

```python
import json
school = {
    "school": "UC Berkeley",
    "address": {
        "city": "Berkeley",
        "state": "California",
        "postal": "94720"
    },
    "list": ["student 1", "student 2", "student 3"]
}
json.dump(school, open('school.json', 'w'))
```

- **Reading from a JSON File**
  - JSON can be read using json.load():

```python
my_school = json.load(open('school.json', 'r'))
print(my_school)
```

  - The loaded object is a dictionary-like structure, allowing easy access to keys and values.

HDF5 Files in Python

- **What is HDF5?**
  - HDF5 (Hierarchical Data Format) is a powerful, open-source binary file format designed for handling large amounts of data. It allows fast reading/writing and supports parallel I/O.
  - HDF5 files store two main types of objects:
    - **Datasets**: Array-like collections of data (similar to NumPy arrays).
    - **Groups**: Folder-like structures that can contain datasets or other groups.
  - HDF5 supports attributes to describe properties of datasets and groups, creating a hierarchical organization similar to a filesystem.

- **Why use HDF5?**
  - No size limit for data storage.
  - Efficient storage and retrieval through built-in optimizations.
  - Commonly used in scientific computing for managing large datasets.

- **Python Support for HDF5:**
  - Libraries: **h5py** and **PyTables**.
  - **h5py** allows Python users to create, read, and manipulate HDF5 files with ease.

Practical Example – Storing Data in HDF5 Format

- **Scenario**: Monitoring accelerations and GPS locations in the Bay Area.
  - **Data**:
    - Two accelerometers (Berkeley and Oakland) with different sampling rates.
    - One GPS station in San Francisco sampling every 60 seconds.
- **Steps for Creating HDF5 File**:
  - Generate random data using NumPy for sensors in different locations.
  - Create an HDF5 file (station.hdf5), and organize the data in groups and subgroups (/acc/1/data, /gps/1/data).
  - Use **attributes** to store additional metadata (e.g., location, start time, sampling interval).

- **Sample Code**:

```python
import numpy as np
import h5py

# Data generation
acc_1 = np.random.random(1000)
location_1 = 'Berkeley'
start_time_1 = 1542000276
dt_1 = 0.04

hf = h5py.File('station.hdf5', 'w')
hf['/acc/1/data'] = acc_1
hf['/acc/1/data'].attrs['dt'] = dt_1
hf['/acc/1/data'].attrs['start_time'] = start_time_1
hf['/acc/1/data'].attrs['location'] = location_1
hf.close()
```

Reading HDF5 Files

- **How to Read Data from HDF5**:
  - Open the file in read mode ('r').
  - Use the keys() function to explore the structure of the file (e.g., see the groups acc and gps).
  - Access datasets directly using paths (e.g., hf_in[‘/acc/1/data']).

- **Example**:

```python
hf_in = h5py.File('station.hdf5', 'r')
acc_data = hf_in['/acc/1/data'][:10]  # Access first 10 entries
acc_attrs = hf_in['/acc/1/data'].attrs['location']  # Access metadata
print(acc_data, acc_attrs)  # Output data and location
```

- **Key Benefits**:
  - Structured organization of data, making it easy to navigate large datasets.
  - Metadata retrieval through attributes enhances data clarity.